CS671D Kaggle Project Report

Zetong Pan

November 26, 2024

1 Exploratory Analysis

From the listed info, the dataframe contains numerical, boolean, datetime, and object features. To exam the features, we devide them into different groups by their Dtypes. In addition, some features contain null values, which are needed to be imputed so that we can estimate the overall relationship between the specific feature and y.

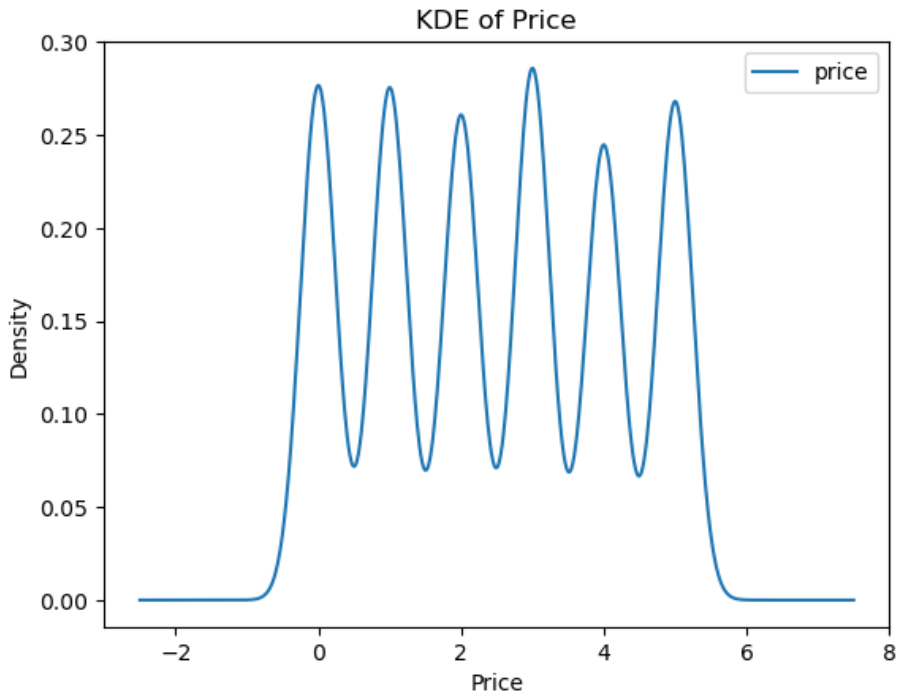
```
 #   Column                                        Non-Null Count  Dtype
---  ------                                        --------------  -----
 0   name                                          15696 non-null  object
 1   description                                   15309 non-null  object
 2   property_type                                 15696 non-null  object
 3   price                                         15696 non-null  int64
 4   neighbourhood_cleansed                        15696 non-null  object
 5   neighbourhood_group_cleansed                  15696 non-null  object
 6   latitude                                      15696 non-null  float64
 7   longitude                                     15696 non-null  float64
 8   host_since                                    15696 non-null  datetime64[ns]
 9   host_response_time                            13493 non-null  object
 10  host_response_rate                            13493 non-null  float64
 11  host_acceptance_rate                          13643 non-null  float64
 12  host_is_superhost                             15445 non-null  object
 13  host_listings_count                           15696 non-null  float64
 14  host_total_listings_count                     15696 non-null  float64
 15  host_verifications                            15696 non-null  object
 16  host_has_profile_pic                          15696 non-null  bool
 17  host_identity_verified                        15696 non-null  bool
 18  calculated_host_listings_count                15696 non-null  int64
 19  calculated_host_listings_count_entire_homes   15696 non-null  int64
 20  calculated_host_listings_count_private_rooms  15696 non-null  int64
 21  calculated_host_listings_count_shared_rooms   15696 non-null  int64
 22  room_type                                     15696 non-null  object
 23  accommodates                                  15696 non-null  int64
 24  bathrooms                                     15693 non-null  float64
 25  bathrooms_text                                15686 non-null  object
 26  bedrooms                                      15662 non-null  float64
 27  beds                                          15612 non-null  float64
 28  amenities                                     15696 non-null  object
 29  has_availability                              15578 non-null  object
 30  availability_30                               15696 non-null  int64
 31  availability_60                               15696 non-null  int64
 32  availability_90                               15696 non-null  int64
 33  availability_365                              15696 non-null  int64
 34  instant_bookable                              15696 non-null  bool
 35  minimum_nights                                15696 non-null  int64
 36  maximum_nights                                15696 non-null  int64
 37  number_of_reviews                             15696 non-null  int64
 38  number_of_reviews_ltm                         15696 non-null  int64
 39  number_of_reviews_l30d                        15696 non-null  int64
 40  first_review                                  11222 non-null  datetime64[ns]
 41  last_review                                   11222 non-null  datetime64[ns]
 42  review_scores_rating                          11222 non-null  float64
 43  review_scores_accuracy                        11222 non-null  float64
 44  review_scores_cleanliness                     11222 non-null  float64
 45  review_scores_checkin                         11222 non-null  float64
 46  review_scores_communication                   11222 non-null  float64
 47  review_scores_location                        11221 non-null  float64
 48  review_scores_value                           11222 non-null  float64
 49  reviews_per_month                             11222 non-null  float64
 50  reviews                                       11222 non-null  object
dtypes: bool(3), datetime64[ns](3), float64(17), int64(15), object(13)
```

1.1 Numerical and Datetime Data

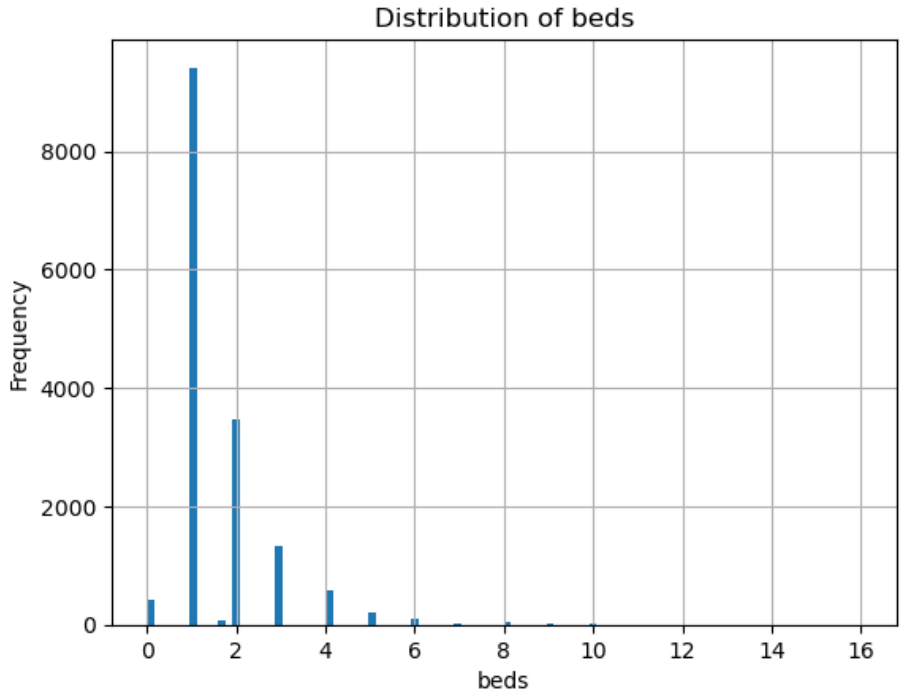
We first select all numerical data inlcuding int64 and float64 and perform mean imputation on NaN values. For Datetime data, we split each into 3 new features of Year, Month, and Day, which are all numerical values. The orignal Datetime

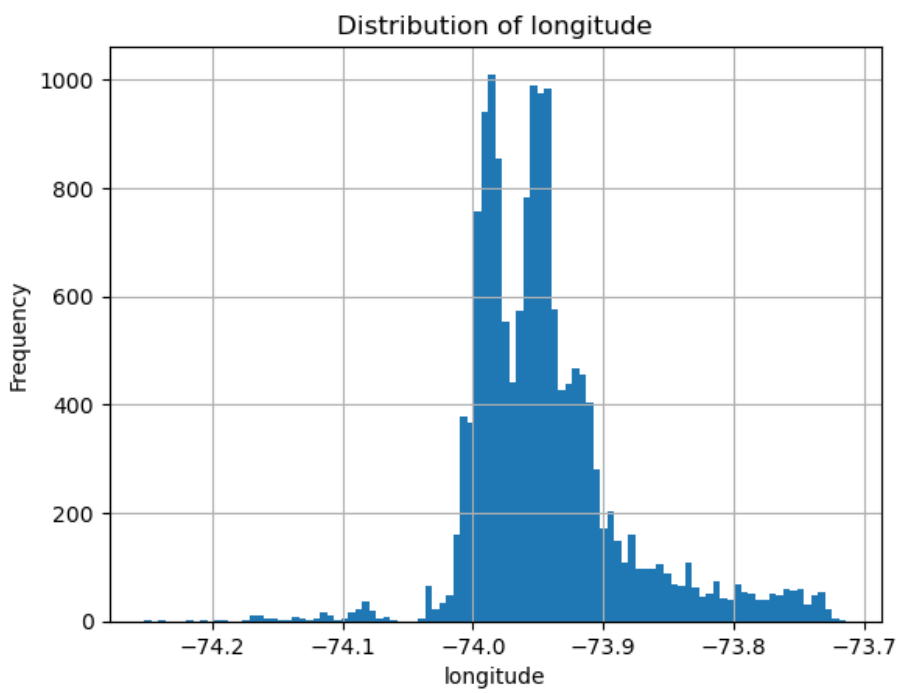
features are dropped and we combine the numerical and new Datetime features together into a new dataframe of numerical features.

We first exam the distribution of y and each features. The KDE of Price shows that the training data we have is pretty balanced, meaning we can just use all data points and do not need to do further modification on this part.

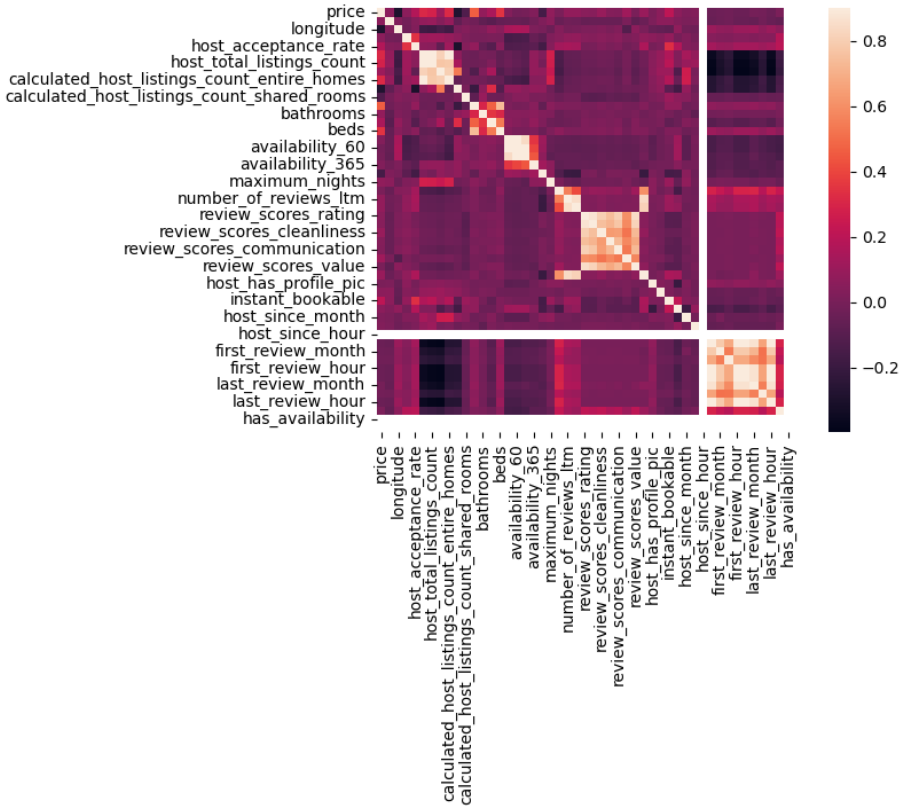


We then plot the distribution of features. 2 examples of the distributions of the features are listed below. Many other features (please see at the appended jupyter notebook), like the 2 listed here, are either roughly in normal distribution or heavily skewed, and they may have a wide range of values. One approach to address these problem is to use tree based algorithms like XGBoost Regressor as they are not sensitive to the distribution of the features. The other approach is simply doing normalization.


Distribution of beds
Frequency
beds
0
2000
4000
6000
8000
0
2
4
6
8
10
12
14
16


Distribution of longitude
Frequency
longitude
0
200
400
600
800
1000
−74.2
−74.1
−74.0
−73.9
−73.8
−73.7

We then use correlations between each feature and y to estimate what are the potential features that can be dropped. We set the threshold to be 0.005: any feature of which the absolute value of correlation with y being below this threshold will be put into a droppable list.



1.2 Boolean and Object Data

We believe name and description are irrelevant to the price of the airbnb, so we simply drop them. In addition, we believe the numerical features have already captured necessary information related to reviews (like ratings, number of reviews), so we do not need to analyze the list of reviews one by one in details. Therefore, we directly drop name, description, and reviews. For all object data but ammenities, we use pd.get_dummies() to one hot encode them. For amenities, we use MultiLabelBinarizer to encode this feature. By combining these dataframes, we get a new dataframe of categorical features. We perform mutual info regression on these features with the price and put any features of which the score being below the threshold into the droppable list.

1.3 Feature Engineering and Selection

We added some interactions between features of which the combination may have meaningful impact on the performance of the model by using addition, subtraction, multiplication, and division. Some newly added features are pretty self explained. The first approach is with review scores as it represent of customers' satisfaction with the house. The second approach is with the availability since everyone is likely to book great stays. The last approach is with the house itself. For example, if a house has many bathrooms comparing with bedrooms, it tends to be luxury. On the contrary, if many bedrooms share one bathroom, it tends to be more economic. In addition, we also try to use latitude and longitude to group data points that are close to each other. However, these methods does not really enhance the performance of the model.

```
df['reviews_total'] = df['number_of_reviews'] * df['review_scores_rating']
df['min_nights_availability'] = df['minimum_nights'] / (df['availability_365']+1)
df['max_nights_availability'] = df['maximum_nights'] / (df['availability_365']+1)
df['reviews_with_day'] = df['number_of_reviews'] / (df['availability_365']+1)
df['accuracy_value'] = df['review_scores_accuracy'] * (df['review_scores_value'])
df['total_host_listings'] = (
    df['calculated_host_listings_count_entire_homes'] +
    df['calculated_host_listings_count_private_rooms'] +
    df['calculated_host_listings_count_shared_rooms']
)

kmeans = KMeans(n_clusters=100, random_state=42)
df['region'] = kmeans.fit_predict(df[['latitude', 'longitude']])

df['availability_mean'] = df[['availability_30', 'availability_60', 'availability_90', 'availability_365']].mean(axis=1)

df['review_scores_std'] = df[['review_scores_accuracy', 'review_scores_cleanliness',
                              'review_scores_checkin', 'review_scores_communication',
                              'review_scores_location', 'review_scores_value']].std(axis=1)

df['bedroom_bathroom_ratio'] = df['bedrooms'] / (df['bathrooms'] + 1)

df['bed_bedroom_ratio'] = df['beds'] / (df['bedrooms'] + 1)
```

For XGB Regressor, we did not normalize the data nor replace NaN value because we believe XGB Regressor is not seneistive to the distribution of data and is able to handle null value. We simply split the datetime into year, month, and day, and encoded categorical data. We first trained model with all features and with selected features which have high correlation with the price. The experiment results show that XGB Regressor tend to have better performance with just all features. We then use model trained with all features to obtain feature importance. We select a list of features that contributed the 95% percent importance to the model for further training. Below is a figure showing the final feature importance, which is pretty intuitive correct as type of rooms/property could has huge impact on the price: the price of a room and a house would be quite different. In addition, price of shampoo or soap could not have a wide range comparing with price of property and thus is not that important.

	feature	importance	cumulative_importance
234	room_type_Entire home/apt	0.360313	0.360313
236	room_type_Private room	0.254357	0.614671
19	minimum_nights	0.065963	0.680633
79	property_type_Room in hotel	0.028858	0.709491
78	property_type_Room in boutique hotel	0.018248	0.727739
...	...	...	...
576	Dr Woods body soap	0.000000	1.000000
577	Dr. Brenners body soap	0.000000	1.000000
578	Dr. Bronner shampoo	0.000000	1.000000
579	Dr. Bronner's Pure Castile Soap body soap	0.000000	1.000000
1349	your head will be so pleased. body soap	0.000000	1.000000

For MLP, we believe the neural network will be able to approximate the nuance relationship between features. Therefore, besides adding a few interactions, we simply feed the model with all features. All features are turned into numerical type: categorical data are encoded and booleans are transformed into 1 and 0. The numerical data are then performed by min-max normalization. We recorded every related values with this normalization from fitting the training data and then applied them on validation set to avoid data leakage.

2 Models

We use XGBRegressor and MLP to predict the price of airbnb.

XGBRegressor is chosen due to the divert distribution of the data points. Besides xgboost itself, helper functions from sklearn, numpy, matplot, and sns are also used.

In addition to XGBRegressor, we also implemented MLP in the hope of the neural network is able to approximate the nuanced relationship between the features and price given abundant training data of which the total number is 15696.

3 Training and Hyperparameter Selection

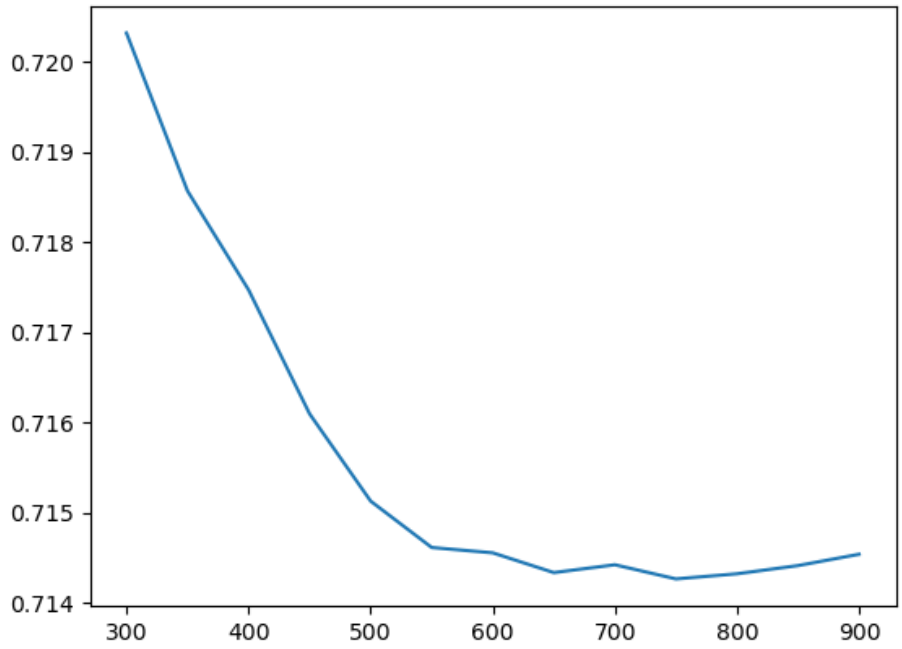
We train the MLP and XGBoost separately.

The MLP uses Adam to perform back propogation to optimize its parameters. Adam basically combines Momentum and RMSProp to do the gradient descent. The RMSProp, which keeps the average of squared gradients, help to find the adaptive (suitable) learning rate under different magnitude of gradients.

It also uses momentum to continue previous iterations's movement, which helps to pass small local minimum and speed up the convergence. The complexity of this model and the way how model is learning could affect the performance. The complexity is basically related to the number of hidden layers as well as the number of perceptrons in each layer. We believe for this project, simple is better.Therefore, we tried the number of layer to be 1 and 2, and 2 was found to be better. In addition, we set the number of perceptrons for each layer equal to 1.5 times the feature size. The process of learning could be affected by batch size, learning rate, and regularization. We choose the batch size to be 16 because this number balances the converging and training speed. As for the learning rate, we tried a few number and found 0.001 to be the optimal. Since we have a great number of features, lasso regularization could be essential especially in minimizing overfitting. As a result of trying different values, we found the number 0.0001 to be the optimal value. For each training, it would take less than 5 minutes to run.

eXtreme Gradient Boosting Regressor is a tree based gradient boosting algorithm which combines many trees (estimators) to make prediction. It is a combination of weak classifiers (trees) to predict the final value. It uses gradient boosting to build these classifiers. Each tree is generated one by one in order to maximize the similarity scores gains calculated with residuals from previous model's threshold. The prediction is modified by the product of the learning rate and weak classifier's prediction. In addition, lasso and ridge regularization could help to combat the overfitting. For this model, we use 2 level training, which is basically doing the training twice. During each level of training, we have 2 ways of evaluation. The first one is to use GridSearchCV with 5 folds, which helps to automatically find the optimal set of parameters from a parameter grid. For the number of estimators, we simply run experiments from 200 to 900 and found the optimal value varies from 500 to 900. As a result, we choose 750 (though the optimum value varies with experiment especially if other parameters are changed and we usethe set of hyperparameters provided by GridSearchCV). Below is an example of figure showing the relationship between number of estimators and the validation loss. Considering there are hundreds of estimators and the price level is within the range between 0 and 5, we choose 0.01, 0.05, 0.1 as the learning rate for experiments and found 0.05 to be the optimum. Since there are many features, lassso regularization could also be essential. The GridSearchCV shows that 0.1 would be optimal than either 0 or 0.2. Other parameters including but not limited to max depth, subsample, gamma are also evaluated by GridSearchCV. GridSearchCV is easy to utilize. However, it measures the RMSE of continuous predicted values from the regressor. Therefore, the second way of evaluation is to use train test split from sklearn and manually fit, predict and calculate the RMSE for rounded discrete predictions. Basically GridSearchCV helps to find the optimal set of parameters and the second one gives an estimation of how good the actual result is. Two ways of evaluation is used in 2 levels of training. The first level of training tried using all features and only features with high correlation. The result shows that all features just have better performance. We than use the set of parameters

from this training and all features to train a model, which is used to check feature importance. For the second level of training, we only select features that has high contribution to the importance for training. Although it does help to reduce the fitting speed, it does not increase the performance much. Overall, it would only take a few seconds to run the fitting.



4 Data Splits

The MLP and XGBRegressor are done in different ways.

The MLP simply uses self written train test split function to split the data.

The XGBRegressor has 2 different ways of evaluation. The first one is using GridSearchCV to tune the hyperparameters. A 5 folds cross validation is used in this process, which is automatically done by this function. For the second evaluation, we simply uses train test split function from sklearn to split the data.

First, as there is not data leakage, the validation loss can be a metric to determine if the model is overfitting. In addition, the result of validation is cross checked with the test result from Kaggle submission. As these 2 numbers are close, the model could not be overfitting..

5 Reflection on Progress

During training, the validation loss at first is always obviously higher than that on Kaggle submission. We were first thinking it could be due to overfitting. However, later we found the Kaggle is evaluating rounded results while we are evaluating continuous prdictions locally. The harded part could be choosing features as the number of features exploded after encoding categorical data. To address this, we use correlation check and feature importance as described before.

6 Predictive Performance

User ID: zetongpan

7 Final Result

Since the experiments shows MLP has wrose results comparing with the XGBRegressor for the validation loss, we use predictions from XGBRegressor for submission to the Kaggle Competition. The code of MLP and XGBregressor is attached at the back of this report. As many parameters have been tried, the parameters as well as training stats shown on the XGB's jupyter notebook or MLP's python file may not be consistent with the analysis above or the set of parameters used for submission to the competition. Yet it shows how we run the experiments to find the parameters and features to make the best prediction.

8 Code

Please see code at: https://github.com/panzertonBirdsong/cs671d_kaggle_project.git